PHYSICAL AI MISSION LAB

Silicon Technician

AI-Assisted Data-Center Fault Diagnosis

A simulation-driven mission lab for inspecting telemetry, identifying infrastructure anomalies, and supporting human-led technical decisions.

Project at a glance

Author	Umar Tabbsum
Affiliation	Independent Researcher
Focus	Physical AI, infrastructure diagnostics, operational simulation
Model	NVIDIA Nemotron-3-Ultra-550b-a55b via Nebius Token Factory
Demo	https://silicon-technician-judge-api.onrender.com/
Video	https://youtube.com/watch?v=KoFntgl08Zk



This project demonstrates an AI-assisted diagnostic workflow in a simulated environment. It does not directly control physical servers, robots, or data-center equipment.

Executive Summary

Modern data centers depend on reliable power, cooling, host scheduling, and hardware telemetry. When several signals change at once, a technician must distinguish a critical fault from a warning, identify the evidence, and determine what should be checked next.

Silicon Technician: Physical AI Mission Lab is a browser-based prototype for exploring that workflow. It combines a simulated data-center mission environment, structured telemetry, deterministic inspection logic, and an AI diagnostic co-pilot powered by NVIDIA Nemotron through Nebius Token Factory.

The prototype lets a user inspect simulated racks, select fault scenarios, review evidence, request a model-generated diagnosis, and export results. A recorded live inference demonstrates the model identifying a simulated Rack B fan failure as critical. The interface also provides a separate recorded or deterministic path so a judge can explore the experience without repeatedly making paid model requests.

Core contribution. The project connects AI-assisted reasoning with a practical infrastructure inspection workflow. It emphasizes evidence, severity, next-step guidance, and human review rather than allowing a language model to autonomously operate equipment.

Scope boundary. The mission lab is a software prototype. Rack telemetry and recovery outcomes are simulated, and the evaluation results in this report are internal tests using generated cases. They do not establish real-world diagnostic accuracy or demonstrate physical actuation.

Contents

Executive Summary i

1 Problem Statement 1

1.1 Project objectives . . . 1

1.2 Why this relates to Physical AI . . . 1

2 System Architecture 1

2.1 Browser mission interface . . . 2

2.2 Diagnostic service . . . 2

2.3 Telemetry and host observations . . . 2

2.4 Human-in-the-loop design . . . 2

3 Model and Inference Infrastructure 2

3.1 NVIDIA Nemotron . . . 2

3.2 Role of Nebius Token Factory . . . 3

3.3 Why use a language model? . . . 3

3.4 Deterministic checks and model output . . . 3

4 Mission Lab Scenarios 3

4.1 Repeatable inspection . . . 4

4.2 Export and review . . . 4

5 Case Study: Rack B Fan Failure 4

5.1 Interpretation . . . 4

5.2 Simulated recovery walkthrough . . . 5

6 Internal Evaluation 5

6.1 Generated case results . . . 5

6.2 Visual comparison . . . 6

6.3 What the results do and do not show . . . 6

6.4 Next evaluation steps . . . 6

7 Host Scheduling Observation 6

8 Physical AI Relevance and Review Alignment 7

8.1 How the project fits the track . . . 8

9 Engineering Decisions and Safety 8

9.1 Separating recorded and live inference . . . 8

9.2 Evidence before action . . . 8

9.3 Required production safeguards . . . 8

10 Limitations and Future Work 8

10.1 Current limitations . . . 9

10.2 Roadmap . . . 9

11 Conclusion 9

References and Project Links 11

A Terminology 11

B Reproducibility Notes 11

C Author Statement 12

1 Problem Statement

Data-center incidents can involve overlapping symptoms: abnormal temperatures, failed fans, power-supply irregularities, or host-level contention. A monitoring dashboard can expose measurements, but an operator still needs to interpret the signals and decide what to investigate.

The prototype explores four practical questions:

1. Which simulated component appears to be in an abnormal state?
2. What telemetry supports the diagnosis?
3. How urgent is the issue, and what should a technician verify next?
4. How can a model assist without replacing operational controls or human authorization?

1.1 Project objectives

- Present a visual mission environment with selectable rack scenarios.
- Combine structured signals with a model-generated diagnostic summary.
- Keep the diagnosis tied to observable evidence and explicit limitations.
- Provide an inspectable, repeatable judge walkthrough.
- Separate simulation and advisory behavior from any real hardware action.

1.2 Why this relates to Physical AI

Physical AI systems operate in or interact with physical environments. Data-center infrastructure is a physical system with thermal, electrical, mechanical, and computational behavior. This project approaches that environment through a simulation and diagnostics layer.

It is best described as a **Physical AI-oriented diagnostic prototype**, not as a deployed robotics system. The current implementation does not include a robot, a physical actuator, or a live connection to production data-center equipment.

2 System Architecture

The prototype is organized into a browser-based mission interface and a Python diagnostic service. Simulated telemetry supplies the environment state. Deterministic logic supports inspection and scenario handling, while the hosted language model generates a diagnostic response when the user explicitly invokes live inference.

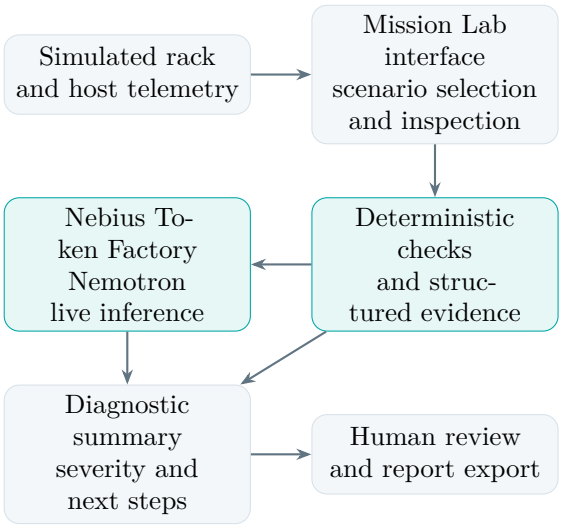


Figure 1: Conceptual workflow. The model is an advisory component; it is not an actuator controller.

2.1 Browser mission interface

The front end provides a visual environment for selecting scenarios and reviewing simulated system conditions. The workflow is designed to be demonstrable without requiring a physical data center.

2.2 Diagnostic service

The Python service prepares diagnostic context and communicates with the hosted inference endpoint when live inference is requested. The application uses structured input and response handling to make the output easier to inspect. The model is not fine-tuned as part of this project.

2.3 Telemetry and host observations

The prototype includes synthetic rack telemetry and a host scheduling experiment using Linux `/proc/schedstat`. The latter is a separate container-level observation, not a measurement of a production GPU server.

2.4 Human-in-the-loop design

The operator chooses a scenario, initiates inspection, reviews the evidence, and decides what to do with the result. The application does not autonomously execute repairs or issue hardware commands.

3 Model and Inference Infrastructure

3.1 NVIDIA Nemotron

The project uses the hosted model identifier:

```
nvidia/Nemotron-3-Ultra-550b-a55b
```

The model is accessed through Nebius Token Factory. In this prototype, the model serves as a diagnostic co-pilot: it receives a structured description of the selected scenario and produces a textual assessment.

The implementation uses prompt engineering, structured context, response validation, and error handling. It does not fine-tune model weights or claim that the model was trained on the project's synthetic scenario data.

3.2 Role of Nebius Token Factory

Nebius Token Factory provides the hosted inference interface used by the application. This allowed the prototype to experiment with a large language model without provisioning a dedicated GPU server or building a custom model-serving cluster.

> The infrastructure claim is intentionally narrow: the application calls a hosted inference API. This report does not claim ownership of, or direct deployment onto, the underlying GPU infrastructure.

3.3 Why use a language model?

A diagnostic model can help organize a collection of signals into a readable explanation. In this project, its potential value is in summarizing evidence, explaining why a scenario may be critical, and proposing verification steps.

However, generated text can be incomplete or incorrect. A model output must not be treated as proof that a component has failed, nor as authorization to execute a repair.

3.4 Deterministic checks and model output

The project separates deterministic scenario behavior from model-generated interpretation. This separation helps the demo remain repeatable and makes it possible to compare a model response with a known synthetic scenario.

It does not eliminate model error. A production implementation would need independent validation, calibrated alert thresholds, access controls, audit logging, and testing against real operational data.

4 Mission Lab Scenarios

The interface supports several simulated conditions designed to exercise different diagnostic paths.

Scenario	Simulated evidence	Intended inspection focus
Rack B fan failure	Fan speed reported at 0 RPM with CPU temperature at 91°C and thermal headroom at 9°C	Cooling fault, severity, and verification steps
Rack C hot inlet	Simulated inlet temperature of 39°C	Airflow and environmental conditions
Rack A PSU anomaly	Simulated power-supply anomaly	Power telemetry and component verification
Healthy baseline	Normalized baseline signals	Compare ordinary conditions with fault scenarios

Table 1: Mission scenarios and their intended diagnostic focus.

4.1 Repeatable inspection

The user can choose a scenario and inspect the relevant simulated state. The interface supports a guided walkthrough, voice narration controls, and report export. These features make it easier for reviewers to examine the project without needing access to a physical rack.

4.2 Export and review

The application supports exporting diagnostic information in formats such as JSON and text. Exported results are useful for reviewing the scenario and its generated summary, but they should not be interpreted as a validated maintenance record for real equipment.

5 Case Study: Rack B Fan Failure

One recorded live inference used a simulated Rack B fan failure. The model classified the situation as **CRITICAL**, consistent with the synthetic telemetry provided to it.

Recorded diagnostic evidence

Fan speed	**0 RPM**
CPU temperature	**91°C**
Thermal headroom	**9°C**
Inference latency	**9,817 ms**
Recorded request ID	`01b44486-46c0-4010-9ba8-387264efdf25`

5.1 Interpretation

The combination of zero reported fan speed, high CPU temperature, and limited thermal headroom makes a cooling issue a plausible explanation within the simulated scenario. The model's role is to summarize that evidence and assist the operator in deciding which checks to perform.

The recorded output is evidence that a live model request occurred and returned a critical classification for this scenario. It is not proof of general diagnostic accuracy across real-world failures.

5.2 Simulated recovery walkthrough

The interface also provides a simulated recovery sequence. In the demonstration, CPU temperature changes from 91°C to 64°C, while fan speed changes from 0 RPM to 7200 RPM.

These values represent a demonstration state transition, not a real repair. The prototype does not send a command to a physical fan, and the displayed recovery does not prove that a physical system has been restored to service.

A production system should verify recovery using trusted telemetry from the actual hardware and require authorized operator approval before any control action.

6 Internal Evaluation

The project was evaluated using generated diagnostic cases to compare model responses with a fixed-rule baseline. The test sets were created for internal iteration; they are not independent benchmark datasets.

6.1 Generated case results

Generated test set	Fixed-rule baseline	Nemotron	Median model time
20 cases, seed 777	6/20 (30.0%)	20/20 (100%)	Not recorded here
60 cases, seed 2026	17/60 (28.3%)	57/60 (95.0%)	5.4 s
60 cases, seed 3031	20/60 (33.3%)	60/60 (100%)	Not recorded here

Table 2: Internal generated-case results. Results reflect the project's test setup and scoring criteria.

6.2 Visual comparison

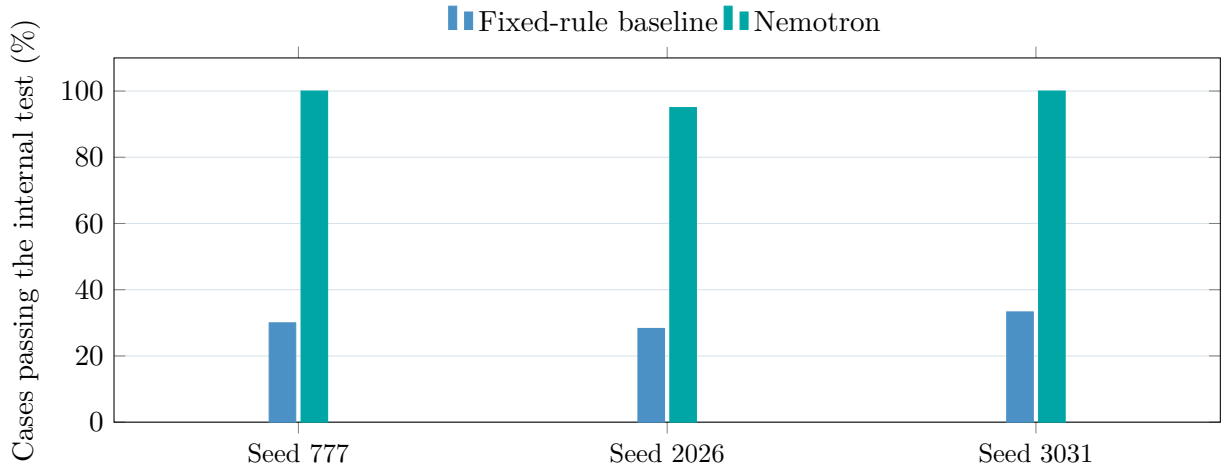


Figure 2: Pass rates on the project's generated test sets. This chart is not a real-world accuracy benchmark.

6.3 What the results do and do not show

The generated tests indicate that the model performed well under the specific scenarios and scoring rules used during development. They are useful for checking whether changes to the prompt or response handling break the intended demo behavior.

The tests do not establish generalization to unseen real incidents. Because the cases are generated and the evaluation is internal, the results may be affected by scenario construction, prompt assumptions, and the definition of a passing answer.

6.4 Next evaluation steps

A stronger evaluation should include:

- A documented, held-out dataset from qualified domain experts.
- Explicit severity and evidence-grounding scoring criteria.
- False-positive and false-negative analysis.
- Repeated tests across model versions and prompt changes.
- Latency and cost reporting under a defined request workload.
- Comparison against a baseline designed with equivalent input information.

7 Host Scheduling Observation

A separate experiment examined Linux host scheduling statistics through `/proc/schedstat` in a container environment. The experiment compared an idle state with a state in which two CPU-bound competing tasks were running.

The recorded scheduling-wait observation for core 0 increased from approximately 43 microseconds in the idle condition to approximately 1,716 microseconds under the tested contention condition, while reported utilization reached 100%.

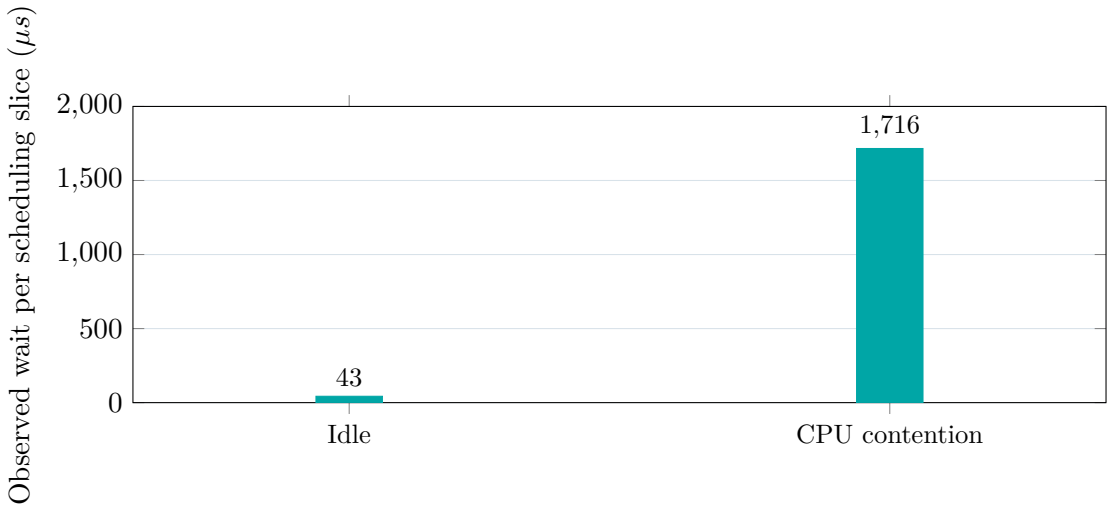


Figure 3: Observed scheduling-wait values from the container-level experiment.

This experiment illustrates how host-level contention can affect scheduling behavior. It is not a measurement of GPU utilization, a dedicated inference server, or production data-center performance. Further work would need repeatable measurement definitions, multiple runs, and controlled hardware.

8 Physical AI Relevance and Review Alignment

The following individuals were selected from the judge list supplied for this report because their listed roles most closely relate to Physical AI, robotics, edge platforms, or systems software.

Name	Listed role	Relevance to the project
Johnny Nunez	AI Developer Advocate – Physical AI & Robotics, NVIDIA	AI systems and Physical AI framing
Evan Helda	Head of GTM Physical AI, Nebius	Physical AI ecosystem and infrastructure context
Dhruv Diddi	Principal – Physical AI Ecosystem, Nebius	Physical AI ecosystem relevance
Aditya Sahu	TME, Jetson Platform, NVIDIA	Edge computing and platform context
Lakshmi Rameish	Senior System Software Engineer, NVIDIA	Systems software and telemetry context

Table 3: Relevance-based selection from the user-provided judge list.

This selection is intended only to explain the project's topical alignment. It does not imply that these individuals are assigned reviewers, have evaluated the project, or endorse its approach.

8.1 How the project fits the track

The strongest connection to Physical AI is the use of a simulated physical infrastructure environment to explore inspection, diagnosis, and human-guided operational decisions. The project offers a starting point for integrating real telemetry and, eventually, validated digital-twin or robotics workflows.

The present prototype should not be described as a fully deployed autonomous Physical AI system. Its current contribution is the diagnostic workflow and simulation experience.

9 Engineering Decisions and Safety

9.1 Separating recorded and live inference

Live model requests can incur usage costs. For that reason, the project provides a recorded or deterministic exploration path alongside the explicit live-inference path. This helps reviewers inspect the interface without unintentionally repeating model requests.

9.2 Evidence before action

The interface is designed around a sequence of observation, interpretation, and review. A production deployment would need to ensure that any generated recommendation can be traced to trusted signals and that an authorized technician remains responsible for decisions.

9.3 Required production safeguards

Before use in a real data center, the system would need:

- Authenticated and authorized access to telemetry.
- Secure handling of credentials and sensitive infrastructure data.
- Input validation and strict schema checks.
- Model-output validation and protection against unsupported claims.
- Audit logs for observations, recommendations, approvals, and actions.
- Human approval for maintenance or control actions.
- Independent safety testing and clear escalation procedures.
- Validation against real hardware, real failure modes, and expert-reviewed cases.

No autonomous repair. The prototype does not control physical actuators, switch power supplies, change fan speeds on real hardware, or establish that a real rack is safe to return to service. Simulated state changes must remain clearly labeled as simulated.

10 Limitations and Future Work

10.1 Current limitations

- The rack environment and scenario telemetry are simulated.
- The model is accessed through a hosted API and is not fine-tuned here.
- Generated test cases are not an independent real-world benchmark.
- The host scheduling experiment is container-level and limited in scope.
- There is no direct integration with a physical robot, camera, BMC, or production data-center control plane.
- Simulated recovery does not verify a real repair.

10.2 Roadmap

1. **Telemetry integration:** connect to an authorized test source using read-only access and documented schemas.
2. **Evidence grounding:** attach each diagnostic claim to specific measurements, thresholds, timestamps, and source identifiers.
3. **Evaluation:** build an expert-reviewed dataset with reproducible train, validation, and held-out splits where applicable.
4. **Digital-twin validation:** compare simulated system behavior against a controlled test environment.
5. **Operational safeguards:** add role-based access, audit trails, approval gates, and rollback planning before considering any control actions.
6. **Edge exploration:** investigate edge-device integration only after the software workflow and safety requirements are validated.

11 Conclusion

Silicon Technician: Physical AI Mission Lab explores how a hosted language model can assist a technician-oriented diagnostic workflow in a simulated data-center environment. It combines a visual mission interface, synthetic telemetry, deterministic scenario handling, and an optional Nemotron inference path through Nebius Token Factory.

The recorded Rack B example demonstrates the intended interaction: the system receives a simulated fan-failure scenario and returns a critical diagnostic assessment. The generated-case tests provide an initial development signal, but they are not evidence of real-world reliability.

The next meaningful step is not to claim autonomy. It is to strengthen the connection between recommendations and verified evidence, test against realistic expert-reviewed cases, and introduce secure read-only telemetry integration with clear human authorization boundaries.

Project principle

Use AI to help people interpret complex system evidence. Keep the evidence inspectable, the

limitations visible, and consequential decisions under authorized human control.

References and Project Links

The links below are starting points for verifying model and infrastructure details. Model specifications and service availability can change; consult the official source for the current description.

[1] **NVIDIA Nemotron model card.** https://build.nvidia.com/nvidia/nemotron-3-ultra-550b-a55b/modelcard

[2] **NVIDIA Nemotron research.** https://research.nvidia.com/labs/nemotron/Nemotron-3/

[3] **Nebius Token Factory.** https://nebius.com/services/token-factory

[4] **Nebius Token Factory cookbook, Nemotron.** https://github.com/nebius/token-factory-cookbook/blob/main/models/nemotron/nemotron3-ultra-550b-a55b.md

[5] **Interactive project demo.** https://silicon-technician-judge-api.onrender.com/

[6] **Project demonstration video.** https://youtube.com/watch?v=KoFntgl08Zk

A Terminology

Physical AI
AI systems connected to or reasoning about physical environments, including robotics, edge systems, simulation, and embodied workflows.

Telemetry
Measurements and status signals collected from a system, such as temperature, fan speed, power readings, or host performance counters.

Inference
Running a trained model to produce an output from supplied input.

Human-in-the-loop
A workflow in which a person reviews or authorizes consequential decisions rather than allowing a model to act without oversight.

Digital twin
A digital representation of a physical system. A simulation alone is not necessarily a validated digital twin.

Generated-case evaluation
An internal test using constructed cases and predefined scoring rules. Its results should not be assumed to generalize to real incidents.

B Reproducibility Notes

For a reproducible submission, retain the source code, the scenario definitions, the evaluation scripts, and the exact scoring criteria used for each reported test set. Record the model identifier, inference date, prompt version, and relevant configuration for every live evaluation.

Do not publish credentials, API keys, private endpoint tokens, or sensitive infrastructure information in the report or repository.

C Author Statement

Umar Tabbsum is the independent researcher behind this prototype. The work explores a human-supervised approach to AI-assisted infrastructure diagnosis, using a simulated environment to make the workflow inspectable and repeatable.